

```
:- [lab6lmc3].
```

```
%%%%%%%%%Exercitiul 1:
```

```
l2x13(L,Ord) :- L=[0,a,b,c,d,1],  
               orddinsucc([(0,a),(0,b),(a,c),(b,c),(b,d),(c,1),(d,1)],L,Ord).
```

```
l2x12plus12(L,Ord) :- L=[0,u,v,w,1],  
                    orddinsucc([(0,u),(0,v),(u,w),(v,w),(w,1)],L,Ord).
```

```
fctlestrcresc(P,OrdP,Q,OrdQ,LF) :-  
    setof(F, fctstrcresc(F,P,OrdP,Q,OrdQ), LF), !.  
fctlestrcresc(_,_,_,_,[]).
```

```
fctL2xL3laL2xL2plusL2(LF) :- l2x13(P,OrdP), l2x12plus12(Q,OrdQ),  
                             fctlestrcresc(P,OrdP,Q,OrdQ,LF).
```

```
fctlepastrsucc(LF,OrdP,OrdQ) :- succdinord(OrdP,SuccP),  
                                succdinord(OrdQ,SuccQ), write(SuccP), nl, write(SuccQ), nl,  
                                auxfctlepastrsucc(LF,SuccP,SuccQ).
```

```
auxfctlepastrsucc([],_,_).  
auxfctlepastrsucc([F|LF],SuccP,SuccQ) :- afislista([F,SuccP,SuccQ]), nl,  
    pastrel(F,SuccP,SuccQ),  
    write(F), nl,  
    auxfctlepastrsucc(LF,SuccP,SuccQ).
```

```
toatepastreazasucc :- fctL2xL3laL2xL2plusL2(LF), l2x13(_,OrdP),  
                      l2x12plus12(_,OrdQ), fctlepastrsucc(LF,OrdP,OrdQ).
```

```
toatepastrsucc :- fctL2xL3laL2xL2plusL2(LF), afislista(LF), n1,  
                l2x13(_,OrdP), l2x12plus12(_,OrdQ),  
                succdinord(OrdP,SuccP), succdinord(OrdQ,SuccQ),  
                afislista([SuccP,SuccQ]), n1,  
                not((member(F,LF), not(pastrrel(F,SuccP,SuccQ))))).
```

%%%%%%%%%Exercitiul 2:

```
ipoteza1(Alfa,Beta,Gama) :- implica(Alfa, Beta;Gama).
```

```
ipoteza2(Alfa,Beta) :- implica(Beta, not(Alfa)).
```

```
ipoteza3(Alfa,Beta,Gama) :- implica(not(Alfa), (not(Beta),Gama)).
```

```
concluzia(Beta,Gama) :- not(Beta), Gama.
```

```
regded :- not((listaValBool([Alfa,Beta,Gama]), ipoteza1(Alfa,Beta,Gama),  
                        ipoteza2(Alfa,Beta), ipoteza3(Alfa,Beta,Gama),  
                        not(concluzia(Beta,Gama))))).
```

%%%%%%%%%Exercitiul 3:

```
multA([a,b,c,d]).
```

```
succA([(a,b),(c,d)]).
```

```
posetA(MultElemA,OrdA) :- multA(MultElemA), succA(SuccA),  
    orddinsucc(SuccA,MultElemA,OrdA).
```

```
bijectie(F,A,B) :- permutare(B,P), constrbij(F,A,P).
```

```
constrbij([],[],[]).  
constrbij([(H,FH)|L],[H|T],[FH|U]) :- constrbij(L,T,U).
```

```
izomleposet(P,OrdP,Q,OrdQ,LF) :- setof(F, izomposet(F,P,OrdP,Q,OrdQ), LF), !.  
izomleposet(_,_,_,_,[]).
```

```
izomposet(F,P,OrdP,Q,OrdQ) :- bijectie(F,P,Q), pastrel(F,OrdP,OrdQ),  
    invrel(F,G), pastrel(G,OrdQ,OrdP).
```

```
detf(Fctf) :- posetA(A,OrdA), invrel(OrdA,InvOrdA),  
    izomleposet(A,OrdA,A,InvOrdA,LF), member(Fctf,LF),  
    member((a,Fa),Fctf), member((b,Fb),Fctf),  
    not(member(Fa,[a,b])), not(member(Fb,[a,b]))).
```

```
detR(RelR) :- multA(A), succA(SuccA), eqgen(SuccA, A, RelR).
```

```
verifAsatepsilon :- multA(A), detf(F), detR(R),  
    not((member(X,A), not((member(Y,A),  
    member((X,FX),F), member((Y,FY),F), member((FY,FFY),F),  
    write('f('), write(X), write(')='), write(FX), tab(3),
```

```
write('f(f(', write(Y), write('))='), write(FFY), nl,  
not(implica(member((X,Y),R), member((FX,FFY),R)))))).
```